

Week 7 Practical

Write a program that prints the addresses Of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

```
divya@Divyasree:~$ mkdir memory_layout
divya@Divyasree:~$ cd memory_layout
divya@Divyasree:~/memory_layout$ pwd
/home/divya/memory_layout
divya@Divyasree:~/memory_layout$ nano memory_layout.c
divya@Divyasree:~/memory_layout$ gcc memory_layout.c -o memory_layout
divya@Divyasree:~/memory_layout$ ls
memory_layout.c
divya@Divyasree:~/memory_layout$ ./memory_layout
Linux Process Address Space
=====
Code address : 0x589765f861c9
Global address : 0x589765f89010
Static address : 0x589765f89014
Heap address : 0x58978c51e2a0
Stack address : 0x7ffd2cf329bc
divya@Divyasree:~/memory_layout$ |
```

Code:

```
#include <stdio.h>
#include <stdlib.h>

// Global variable
int global_var = 100;

// Static variable
static int static_var = 200;

// Function to represent code segment
void code_function()
{
    printf("Inside code function\n");
}

int main()
{
    // Local variable - Stack
    int stack_var = 300;

    // Dynamic variable - Heap
    int *heap_var = (int *)malloc(sizeof(int));
    *heap_var = 400;

    printf("Linux Process Address Space\n");
    printf("=====\n");

    printf("Code address : %p\n", (void *)code_function);
    printf("Global address : %p\n", (void *)&global_var);
```

```
printf("Linux Process Address Space\n");
printf("=====\n");

printf("Code address : %p\n", (void *)code_function);
printf("Global address : %p\n", (void *)&global_var);
printf("Static address : %p\n", (void *)&static_var);
printf("Heap address : %p\n", (void *)heap_var);
printf("Stack address : %p\n", (void *)&stack_var);

free(heap_var);

return 0;
}
```

Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```
divya@Divyasree:~$ mkdir proc_memory_analysis
cd proc_memory_analysis
divya@Divyasree:~/proc_memory_analysis$ nano memory_process.c
divya@Divyasree:~/proc_memory_analysis$ gcc memory_process.c -o memory_process
divya@Divyasree:~/proc_memory_analysis$ ./memory_process
Process is running...
PID: 950

Memory Addresses:
Global : 0x6054004cf010
Static : 0x6054004cf014
Heap : 0x60543624e2a0
Stack : 0x7ffe6ea3d05c

Press Enter to terminate...

divya@Divyasree:~/proc_memory_analysis$ cd proc_memory_analysis
-bash: cd: proc_memory_analysis: No such file or directory
divya@Divyasree:~/proc_memory_analysis$ cat /proc/950/maps
cat: /proc/950/maps: No such file or directory
divya@Divyasree:~/proc_memory_analysis$ |
```

Code:

```
GNU nano 7.2 memory_process.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 100;
static int static_var = 200;

int main() {
    int stack_var = 300;

    int *heap_var = malloc(sizeof(int));
    *heap_var = 400;

    printf("Process is running...\n");
    printf("PID: %d\n", getpid());

    printf("\nMemory Addresses:\n");
    printf("Global : %p\n", (void *)&global_var);
    printf("Static : %p\n", (void *)&static_var);
    printf("Heap : %p\n", (void *)heap_var);
    printf("Stack : %p\n", (void *)&stack_var);

    printf("\nPress Enter to terminate...\n");
    getchar();

    free(heap_var);

    return 0;
}
```